

Tester

Varför testar man kod?

- Kod är korrekt
- Kod körs tillräckligt snabbt
- Undvika att introducera nya buggar
- Dokumentation

Olika typer av tester

- Enhetstester/unit tests
- Integrationstester/integration tests
- End-2-end-tester (dörr-till-dörr-tester?)
- Bottom-up

Regression testing

- Man bygger upp en mängd tester under projektets gång
- Förhindrar att buggar återuppstår
- Varje buggfix kan t.ex. ha ett associerat test

Fuzz testing / fuzzing

- Släng slumpmässig input på programmet
- T.ex. kompilatorer/språkverktyg som körs med randomiserad kod
- Exempel: [libFuzzer](#) (LLVM)
- Kan analysera vilka delar av koden som påverkas och generera ny input för att täcka fler delar.

Stress testing

- Testar hur en applikation hanterar onormalt höga belastningar
- T.ex. webbservrar

Benchmarking

- Testar applikationens prestanda
- Nödvändigt för realtidssystem (flygplan etc.)
- Kommer vara krav i en av läxorna

Manuella tester

- Om det är för svårt/tidskrävande att konstruera tester för något
- Demo?

Automatiska tester

- T.ex. GitHub actions
- Tester körs automatiskt för ett pull request
- Om testerna passerar kan PR:en merge:as

Extra

- Bra video om testing finns [här](#).

Tester i Rust

Funktionen som vi testar

```
1 fn min_even(nums: Vec<u64>) -> Option<u64> {  
2     // Return the smallest even number in `nums`,  
3     // or None if there are no even numbers in `nums`  
4     todo!("Implement this");  
5 }
```



Simpelt test

```
1  #[cfg(test)] // Modulen kompileras endast när tester körs
2  mod test {
3      use super::*; // Importera funktioner etc. från föräldramodulen
4
5      // Tester tester tester...
6      #[test]
7      fn multiple_even() {
8          assert_eq!(min_even(vec![9, 10, 11, 12]), Some(10));
9      }
10 }
```



`#[should_panic]`

```
1  #[test]
2  #[should_panic]
3  fn just_panic() {
4      panic!("Aaaaah");
5  }
```



Era schackbibliotek

Hur har det gått?

- Hur har ni representerat brädet/pjäser etc?
- Hur ser API:t ut? Vilka funktioner finns?
- Hur kan man testa koden?

Till nästa övning

- Fortsätt med schackbiblioteket (inga nya repon)
- **Skriv tester**
- Biblioteket ska vara helt klart

Free time